

# รายงานสัปดาห์ที่ 10 (Basic)

## Automating Spreadsheets — Excel & Google Sheets

สรุปเนื้อหา บันทึกผลแล็บ ผลการรันจริง และการแก้ไขข้อบกพร่องด้านการจัดรูปแบบ  
โปรเจกต์: spreadsheet-automator (openpyxl)

### 1. ผลลัพธ์การเรียนรู้ (Learning Outcomes)

- เข้าใจโครงสร้างของไฟล์ Excel ทั้ง workbook, worksheet, row, column และ cell ในมุมมองของการเข้าถึงด้วยโปรแกรม
- ใช้ไลบรารี openpyxl อ่าน เขียน และแก้ไขไฟล์ .xlsx ได้
- ทำงาน ETL (Extract-Transform-Load) กับข้อมูลในสเปรดชีตแบบอัตโนมัติ
- จัดรูปแบบเซลล์และปรับโครงสร้างชีตด้วยโปรแกรม
- รู้จักเครื่องมือและ API สำหรับเชื่อมต่อสเปรดชีตบนคลาวด์อย่าง Google Sheets

### 2. สรุปเนื้อหาบทเรียน

#### 2.1 ทำไมต้องทำสเปรดชีตอัตโนมัติ

งานสเปรดชีตจำนวนมากเป็นงานซ้ำ ๆ เช่น รวมยอด จัดรูปแบบ ทำรายงานประจำเดือน ซึ่งถ้าทำด้วยมือทุกครั้งจะเสียเวลาและเกิดข้อผิดพลาดง่าย โดยเฉพาะเมื่อข้อมูลมีจำนวนมาก การเขียนโปรแกรมให้ทำแทนช่วยให้ได้ผลลัพธ์ที่สม่ำเสมอ ตรวจสอบย้อนหลังได้ และทำซ้ำได้ทันทีเมื่อข้อมูลเปลี่ยน

#### 2.2 โครงสร้างไฟล์ Excel ในมุมมองของโปรแกรม

| ระดับ     | ความหมาย                        | การเข้าถึงด้วย openpyxl                        |
|-----------|---------------------------------|------------------------------------------------|
| Workbook  | ไฟล์ Excel ทั้งไฟล์ (.xlsx)     | openpyxl.load_workbook() / openpyxl.Workbook() |
| Worksheet | แต่ละแท็บ (ชีต) ในไฟล์          | wb.active, wb['ชื่อชีต'], wb.create_sheet()    |
| Cell      | ช่องข้อมูลแต่ละช่อง เช่น A1, B5 | ws['A1'].value, ws.cell(row=R, column=C)       |

#### 2.3 คำสั่งสำคัญของ openpyxl

- อ่าน/เขียนค่า: sheet['A1'].value สำหรับอ่าน และ sheet['A1'] = '■■■■■■■■' สำหรับเขียน
- วนอ่านทีละแถว: sheet.iter\_rows() โดยใส่ values\_only=True เพื่อรับเฉพาะค่า และ min\_row=2 เพื่อข้ามหัวตาราง
- จัดรูปแบบ: ใช้คลาสจาก openpyxl.styles ได้แก่ Font, PatternFill, Border/Side, Alignment

- รูปแบบตัวเลข: `cell.number_format = '#,##0.00'` ทำให้ 2401 แสดงเป็น 2,401.00 โดยค่าที่เก็บจริงยังเป็นตัวเลข ไม่ใช่ข้อความ
- ผสานเซลล์: `sheet.merge_cells()` และปรับความกว้างคอลัมน์ด้วย `sheet.column_dimensions['A'].width`
- บันทึก: `workbook.save('output.xlsx')` — ต้องกำหนดสไลด์ให้ครบก่อนบันทึกเสมอ

## 2.4 Google Sheets (ภาพรวมเชิงแนวคิด)

สำหรับสเปรดชีตบนคลาวด์ ใช้ Google Sheets API ผ่านไลบรารีอย่าง `gspread` ความต่างสำคัญคือต้องยืนยันตัวตนด้วย OAuth 2.0 และจัดการคีย์ให้ปลอดภัย ต่างจากการเปิดไฟล์ในเครื่องที่เข้าถึงได้ทันที แต่แนวคิดการเข้าถึงข้อมูลยังคล้ายกัน คือระบุขีดแล้วอ่าน/เขียนตามพิกัดแถว-คอลัมน์

## 3. สรุปผลการทำแล็บ: Sales Data Processor

โปรแกรมอ่านข้อมูลการขายจาก `data/input_sales.xlsx` คำนวณราคารวมของแต่ละรายการ และยอดรวมทั้งหมด แล้วสร้างไฟล์รายงานใหม่พร้อมจัดรูปแบบ

- โหลดไฟล์ต้นทางด้วย `load_workbook()` พร้อมดักจับกรณีไฟล์ไม่มีอยู่
- สร้างเวิร์กบุ๊กใหม่และลบชีตเปล่าที่ Excel สร้างมาให้อัตโนมัติ แล้วสร้างชีตชื่อ Sales Report
- วนอ่านข้อมูลตั้งแต่แถวที่ 2 โดยข้ามแถวว่างด้วยเงื่อนไข `if not any(row)`
- แปลงชนิดข้อมูลด้วย `int()` และ `float()` ภายใน try-except หากแปลงไม่ได้จะข้ามแถวนั้นพร้อมแจ้งเตือน
- จัดรูปแบบ: หัวตารางพื้นเขียวตัวอักษรขาว แถวยอดรวมพื้นเหลืองตัวหนา ทุกเซลล์มีเส้นขอบ และคอลัมน์จำนวนเงินใช้รูปแบบ `#,##0.00`
- ผสานเซลล์ A ถึง C ในแถวสุดท้ายเพื่อวางป้าย Grand Total: ชิดขวา
- ปรับความกว้างคอลัมน์อัตโนมัติจากความยาวข้อความที่ยาวที่สุดในคอลัมน์นั้น

## 4. ผลการรันจริง (Test Results)

รันคำสั่ง `python main.py` จริงบนเครื่อง โปรแกรมทำงานจบสมบูรณ์ ไม่มี error

```

week10 - PowerShell

PS C:\xampp\htdocs\proj\week10> python main.py
Starting sales data processing from 'data\input_sales.xlsx'...
Workbook 'data\input_sales.xlsx' loaded successfully.
New workbook 'new_workbook.xlsx' created.
Workbook saved to 'data\output_sales_report.xlsx'.
Sales report successfully generated and saved to 'data\output_sales_report.xlsx'.

```

ภาพที่ 1: ผลการรันจากเทอร์มินัล

### 4.1 ตรวจสอบความถูกต้องของการคำนวณ

| Product | Quantity | Unit Price | Total Price | ตรวจสอบ |
|---------|----------|------------|-------------|---------|
|---------|----------|------------|-------------|---------|

|             |   |          |          |                              |
|-------------|---|----------|----------|------------------------------|
| Laptop      | 2 | 1,200.50 | 2,401.00 | $2 \times 1200.50 = 2401.00$ |
| Mouse       | 5 | 25.00    | 125.00   | $5 \times 25.00 = 125.00$    |
| Keyboard    | 3 | 75.99    | 227.97   | $3 \times 75.99 = 227.97$    |
| Monitor     | 1 | 300.00   | 300.00   | $1 \times 300.00 = 300.00$   |
| Printer     | 1 | 150.00   | 150.00   | $1 \times 150.00 = 150.00$   |
| Grand Total |   |          | 3,203.97 | รวมทั้ง 5 รายการ ถูกต้อง     |

ไฟล์ต้นทางมีข้อมูล 6 แถว แต่รายงานมีเพียง 5 รายการ เพราะแถวที่ 5 เป็นแถวว่างที่โจทย์กำหนดให้ใส่ไว้ และสคริปต์ข้ามให้อัตโนมัติ ซึ่งยืนยันว่าเงื่อนไขตรวจแถวว่างทำงานได้จริง

## 5. ข้อบกพร่องที่พบและการแก้ไข

โค้ดต้นฉบับรันผ่านและคำนวณตัวเลขถูกต้องทุกค่า แต่เมื่อเปิดไฟล์ผลลัพธ์ตรวจดูอย่างละเอียด พบข้อบกพร่องด้านการจัดรูปแบบ 2 จุด ซึ่งไม่ทำให้โปรแกรมพัง แต่ทำให้รายงานดูไม่เรียบร้อย

### 5.1 เปรียบเทียบก่อนและหลังแก้ไข

output\_sales\_report.xlsx — ก่อนแก้ไข

| Product Name | Quantity | Unit Price | Total Price |
|--------------|----------|------------|-------------|
| Laptop       | 2        | 1200.5     | 2,401.00    |
| Mouse        | 5        | 25         | 125.00      |
| Keyboard     | 3        | 75.99      | 227.97      |
| Monitor      | 1        | 300        | 300.00      |
| Printer      | 1        | 150        | 150.00      |
| Grand Total: |          |            | 3,203.97    |

ภาพที่ 2: ก่อนแก้ไข — คอลัมน์ Unit Price แสดงเป็น 1200.5 / 25 / 300 ซึ่งไม่เป็นรูปแบบเงิน และช่องป้าย Grand Total ไม่มีเส้นขอบ

output\_sales\_report.xlsx — หลังแก้ไข

| Product Name | Quantity | Unit Price | Total Price |
|--------------|----------|------------|-------------|
| Laptop       | 2        | 1,200.50   | 2,401.00    |
| Mouse        | 5        | 25.00      | 125.00      |
| Keyboard     | 3        | 75.99      | 227.97      |
| Monitor      | 1        | 300.00     | 300.00      |
| Printer      | 1        | 150.00     | 150.00      |
| Grand Total: |          |            | 3,203.97    |

ภาพที่ 3: หลังแก้ไข — Unit Price แสดงเป็น 1,200.50 / 25.00 / 300.00 เหมือนคอลัมน์ Total Price และแถวยอดรวมมีกรอบครบทั้งแถว

### 5.2 รายละเอียดข้อบกพร่อง

| # | อาการ                                                 | สาเหตุ                                                                                                 | การแก้ไข                                                       |
|---|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| 1 | Unit Price ไม่เป็นรูปแบบเงิน แสดง 1200.5 แทน 1,200.50 | โค้ดกำหนด number_format ให้เฉพาะเซลล์ Total Price (คอลัมน์ D) แต่ลืมคอลัมน์ C ซึ่งเป็นจำนวนเงินเช่นกัน | กำหนด number_format = currency_format ให้เซลล์ Unit Price ด้วย |
| 2 | ช่องป้าย Grand Total: ไม่มีเส้นขอบ                    | โค้ดกำหนด font, fill และ alignment ให้เซลล์ป้าย แต่ไม่ได้กำหนด border ต่างจากเซลล์อื่นที่มีครบ         | เพิ่ม grand_total_label_cell.border = thin_border              |

```

จุดที่ 1: จัดรูปแบบสกุลเงินให้คอลัมน์ Unit Price
- output_sheet.cell(row=output_row, column=3, value=unit_price).border = thin_border
+ unit_price_cell = output_sheet.cell(row=output_row, column=3, value=unit_price)
+ unit_price_cell.number_format = currency_format
+ unit_price_cell.border = thin_border

จุดที่ 2: เพิ่มเส้นขอบให้เซลล์ป้าย Grand Total
grand_total_label_cell.font = total_row_font
grand_total_label_cell.fill = total_row_fill
+ grand_total_label_cell.border = thin_border
grand_total_label_cell.alignment = Alignment(horizontal='right', ...)

```

ภาพที่ 4: โค้ดที่แก้ไข — สีแดงคือบรรทัดที่ถูกแทนที่ สีเขียวคือบรรทัดที่เพิ่มเข้าไป

## 5.3 ข้อสังเกตเพิ่มเติมจากการทดลอง

ตอนแรกคาดว่าเซลล์ B และ C ในแถวยอดรวม (ซึ่งถูก merge รวมกับ A) จะต้องกำหนดเส้นขอบและสีพื้นเองด้วย จึงเขียนลูปกำหนดสไตล์ให้เซลล์เหล่านั้นเพิ่ม แต่เมื่อทดลองเปรียบเทียบผลจริงระหว่างมีลูปกับไม่มีลูป หลังบันทึกไฟล์แล้วเปิดใหม่ พบว่าผลลัพธ์เหมือนกันทุกประการ เพราะ openpyxl กระจายเส้นขอบรอบนอกให้ครบทั้งช่วงที่ merge โดยอัตโนมัติอยู่แล้วเมื่อกำหนดเส้นขอบที่เซลล์ซ้ายบน ส่วนสีพื้นนั้น Excel ก็ใช้สีของเซลล์ซ้ายบนแสดงทั่วทั้งช่วง merge อยู่แล้ว จึงตัดลูปดังกล่าวออก เพราะเป็นโค้ดที่ไม่ได้ทำงานจริง (dead code) การเก็บไว้จะทำให้เข้าใจผิดว่ามันจำเป็น

## 6. คำถาม-คำตอบสำคัญ

### 6.1 number\_format เปลี่ยนค่าที่เก็บในเซลล์หรือไม่?

ไม่เปลี่ยน number\_format กำหนดเพียงวิธีแสดงผลเท่านั้น ค่าที่เก็บจริงยังเป็นตัวเลข เช่น เซลล์เก็บ 2401 แต่แสดงเป็น 2,401.00 ข้อดีคือยังนำไปคำนวณต่อใน Excel ได้ตามปกติ ต่างจากการแปลงเป็นข้อความเองด้วย f"{value:, .2f}" ซึ่งจะทำให้เซลล์กลายเป็นข้อความและคำนวณต่อไม่ได้

### 6.2 ทำไมต้องรันจากในโฟลเดอร์โปรเจกต์เท่านั้น?

เพราะ main.py ระบุเส้นทางไฟล์เป็น os.path.join('data', ...) ซึ่งเป็นเส้นทางสัมพัทธ์กับตำแหน่งที่รันคำสั่ง (current working directory) ไม่ใช่ตำแหน่งของไฟล์สคริปต์ ถ้ารันจากโฟลเดอร์อื่นจะหาไฟล์ไม่เจอ วิธีทำให้ทนทานขึ้นคือใช้ os.path.dirname(\_\_file\_\_) เป็นฐานเหมือนที่ทำในแล็บสัปดาห์ที่ 8

### 6.3 สคริปต์ข้ามแถวว่างได้อย่างไร?

ใช้เงื่อนไข `if not any(row): continue` โดย `any()` จะคืนค่า `False` เมื่อทุกค่าในแถวเป็น `None` หรือค่าที่ถือว่าเท็จทั้งหมด จึงข้ามแถวนั้นไป ข้อควรระวังคือถ้าแถวมีข้อมูลจริงแต่เป็นเลข 0 ทั้งแถว `any()` ก็จะมองว่าเป็นแถวว่างและข้ามไปด้วย ซึ่งอาจไม่ใช่สิ่งที่ต้องการในบางกรณี

## 7. บทสรุป

แล็บนี้แสดงให้เห็นว่างานสเปรดชีตที่ปกติต้องทำด้วยมือ ทั้งการคำนวณ การจัดรูปแบบ และการสร้างรายงาน สามารถเขียนโปรแกรมให้ทำแทนได้ทั้งหมดด้วย `openpyxl` ซึ่งได้ผลลัพธ์ที่สม่ำเสมอและทำซ้ำได้ทันทีเมื่อข้อมูลเปลี่ยน สิ่งที่ได้เรียนรู้เพิ่มเติมจากการทำจริงคือ

โปรแกรมที่คำนวณถูกต้องแล้วยังอาจมีข้อบกพร่องด้านการนำเสนอได้

การเปิดไฟล์ผลลัพธ์ตรวจดูทีละเซลล์จึงสำคัญไม่แพ้การตรวจว่าโปรแกรมรันผ่านหรือไม่

และการทดลองเปรียบเทียบผลจริงก่อนสรุปว่าโค้ดส่วนใดจำเป็น

ก็ช่วยไม่ให้เก็บโค้ดที่ไม่ได้ทำงานไว้ในโปรเจกต์

---

อ้างอิง: เอกสารประกอบการสอน "LabX BASIC: Automating Spreadsheets (Excel & Google Sheets)"

หมายเหตุ: ภาพผลการรันสร้างจากข้อความ output จริงที่ได้จากการรันบนเครื่อง และภาพตารางวาดขึ้นจากข้อมูลและรูปแบบที่อ่านได้จากไฟล์ `.xlsx` จริง (ไม่ใช่ภาพจับหน้าจอ OS โดยตรง)